Homework 1 - Task 1

```
# Initialize Otter
import otter
grader = otter.Notebook("hw1-task1.ipynb")
```

eDNA and Conventional Fish Surveys

Based on: McElroy et al. (2020). *Calibrating environmental DNA metabarcoding to conventional surveys for measuring fish species richness.* Frontiers in Ecology and Evolution.

In this set of exercises you'll wrangle and explore a dataset comparing **eDNA metabarcoding** and **traditional survey methods** for detecting fish species across 68 global freshwater lakes.

The dataset has the following key columns:

Column	Description
`author`	First author of the primary study
`area_ha`	Lake area (hectares)
`dna_richness`	Number of fish species detected by eDNA
`trad_richness`	Number of fish species detected by conventional surveys
`dna_only`	Number of species found **only** by eDNA
`trad_only`	Number of species found **only** by conventional surveys
`shared`	Number of species found by **both** methods
`union`	Total unique species (`dna_only + trad_only + shared`)
`marker_cat`	Whether single or multiple genetic markers were used (single vs. multiple eDNA markers)
`gear_cat`	Whether single or multiple conventional survey gear types were used (single vs. multiple)
`total_vol_liter`	Total volume of water sampled (litres)

There is no statistics or ML methods in this notebook. Its goal is to review some core Python data wrangling. Along the way you'll practice core pandas skills: loading data, inspecting structure, filtering,

grouping, sorting, and deriving new columns.

1.

Import `pandas` with the alias `pd` and `numpy` with the alias `np`.

```
# Import necessary libraries
import pandas as pd
import numpy as np
```

2.

Read in the lakes data and store it in a variable called `df`.

```
# Read the dataset
df = pd.read_csv('lakes_data.csv')

# Display the first few rows of the dataset
df.head()
```

	author	year_pub	area_ha	dna_richness	trad_richness	dna_only	trad_only	shared	union	marker_co
0	Civade	2016	545.0	14	18	1	5	13	19	1
1	Doble	2020	3290000.0	92	62	41	11	51	103	4
2	Evans	2017	2.0	15	10	5	0	10	15	3
3	Fujii	2019	9.0	0	7	0	7	0	7	1
4	Fujii	2019	5.0	2	8	0	6	2	8	1

3.

How many rows and columns does `df` have? Store the number of rows in a variable called `rows` and the number of columns in a variable called `cols`.

```
# Check the number of rows and columns in the dataset
rows = df.shape[0]
cols = df.shape[1]
rows, cols
```

```
(68, 17)
```

```
grader.check("q3")
```

q3 passed! 🎉

4.

Display the data type of each column using `df.info()`. Which columns are numeric and which are categorical?

Answer

Based on the output of `df.info()`:

- the numeric columns (dtype `int64` or `float64`) are `year_pub`, `area_ha`, `dna_richness`, `trad_richness`, `dna_only`, `trad_only`, `shared`, `union`, `marker_count`, `field_reps`, `rep_vol_liter` and `total_vol_liter`
- categorical columns (dtype `object`) are `author`, `locus`, `marker_cat`, `gear_cat` and `match_effort`.

```
# Check the data type for each column in the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 68 entries, 0 to 67
Data columns (total 17 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   author           68 non-null     object
 1   year_pub         68 non-null     int64
 2   area_ha          68 non-null     float64
 3   dna_richness     68 non-null     int64
 4   trad_richness    68 non-null     int64
 5   dna_only         68 non-null     int64
 6   trad_only        68 non-null     int64
 7   shared           68 non-null     int64
 8   union            68 non-null     int64
 9   marker_count     68 non-null     int64
 10  locus            68 non-null     object
 11  marker_cat       68 non-null     object
 12  gear_cat         68 non-null     object
 13  match_effort     68 non-null     object
 14  field_reps       68 non-null     int64
 15  rep_vol_liter    68 non-null     float64
 16  total_vol_liter  68 non-null     float64
```

```
dtypes: float64(3), int64(9), object(5)
memory usage: 9.2+ KB
```

5.

Compute summary statistics — mean, median, min, max, and std — for `dna_richness` and `trad_richness`. Store the result in a data frame called `stats` with the following row names `mean`, `median`, `min`, `max`, `std` and column names `dna_richness` and `trad_richness`.

```
# Compute mean, median, min, max and std for dna_richness and trad_richness
stats = pd.DataFrame({
    "dna_richness": [
        df["dna_richness"].mean(),
        df["dna_richness"].median(),
        df["dna_richness"].min(),
        df["dna_richness"].max(),
        df["dna_richness"].std()
    ],
    "trad_richness": [
        df["trad_richness"].mean(),
        df["trad_richness"].median(),
        df["trad_richness"].min(),
        df["trad_richness"].max(),
        df["trad_richness"].std()
    ]
}, index=["mean", "median", "min", "max", "std"])

# Move the index (mean, median, min, max and std) into its own column
display_stats = stats.copy()
display_stats.insert(0, "Summary Statistics", display_stats.index)
display_stats = display_stats.reset_index(drop=True)
display_stats
```

	Summary Statistics	dna_richness	trad_richness
0	mean	12.382353	12.073529
1	median	9.500000	11.000000
2	min	0.000000	1.000000
3	max	92.000000	62.000000
4	std	12.996015	9.692767

```
grader.check("q5")
```

q5 passed! 🙌

6.

How many lakes had eDNA detect **more** species than conventional surveys? How many had conventional surveys detect **more** species? How many were **tied**?

Store your answers in variables called `edna_greater`, `trad_greater`, and `tied`.

Answer

- eDNA detected more species than conventional surveys in 32 lakes.
- Conventional surveys detected more species than eDNA in 24 lakes.
- Both methods detected the same number of species in 12 lakes.

```
# Count how many lakes fall into each category
edna_greater = int((df["dna_richness"] > df["trad_richness"]).sum())
trad_greater = int((df["trad_richness"] > df["dna_richness"]).sum())
tied         = int((df["dna_richness"] == df["trad_richness"]).sum())

print(edna_greater, trad_greater, tied)
```

```
32 24 12
```

```
grader.check("q6")
```

q6 passed! 🌈

7.

Which lake has the **highest total species richness** (`union`)? Store the row as a `pandas.Series` called `top_lake`, selecting only the `author`, `area_ha`, and `union` columns.

```
# Find the lake with the highest total species richness
top_lake = df.loc[df["union"].idxmax(), ["author", "area_ha", "union"]]

top_lake
```

```
author         Doble
area_ha    3290000.0
union            103
Name: 1, dtype: object
```

```
grader.check("q7")
```

q7 passed! 🌟

8.

Create a new column in `df` called `dna_advantage` defined as:

`dna_advantage = dna_richness - trad_richness`

A positive value means eDNA found more species; a negative value means conventional surveys found more. Display the first 5 rows of `author`, `dna_richness`, `trad_richness`, and `dna_advantage`.

```
# Create a new column `dna_advantage`
df["dna_advantage"] = df["dna_richness"] - df["trad_richness"]

# Display the first 5 rows
df[["author", "dna_richness", "trad_richness", "dna_advantage"]].head()
```

	author	dna_richness	trad_richness	dna_advantage
0	Civade	14	18	-4
1	Doble	92	62	30
2	Evans	15	10	5
3	Fujii	0	7	-7
4	Fujii	2	8	-6

```
grader.check("q8")
```

q8 passed! 🍀

9.

Filter the dataframe to only lakes where **both** conditions are true: - `area_ha` is greater than 1000 hectares - `total_vol_liter` is greater than the median `total_vol_liter`

Store the filtered dataframe in a variable called `large_lakes`. Then store the count of matching lakes in `n_large` and their mean `union` richness in `mean_union_large`.

```
# Compute the median total volume sampled across all lakes
median_vol = df["total_vol_liter"].median()

# Filter lakes with area > 1000 ha AND sampling volume above the median
large_lakes = df[(df["area_ha"] > 1000) & (df["total_vol_liter"] > median_vol)]
```

```
n_large = len(large_lakes)
mean_union_large = large_lakes["union"].mean()

print(n_large, mean_union_large)
```

```
7 38.714285714285715
```

```
grader.check("q9")
```

q9 passed! 🍀

10.

Group lakes by `gear_cat` (single vs. multiple conventional survey methods) **and** `marker_cat` (single vs. multiple eDNA markers). Using `gear_cat` as rows and `marker_cat` as columns, compute a pivot table showing the mean `union` (total species richness) and mean `dna_advantage` for each combination. Round values to 2 decimal places and exclude the `unspecified` category from your interpretation.

Store the result in a variable called `pivot`.

Hint — `pivot_table` syntax:

```
df.pivot_table(
    values=[...],      # columns to aggregate
    index=...,         # variable to use as rows
    columns=...,       # variable to use as columns
    aggfunc="mean"     # aggregation function
).round(2)
```

```
# Build a pivot table with gear_cat as rows and marker_cat as columns
pivot = df.pivot_table(
    values=["union", "dna_advantage"],
    index="gear_cat",
    columns="marker_cat",
    aggfunc="mean"
).round(2)

pivot
```

	dna_advantage		union	
marker_cat	**multiple**	**single**	**multiple**	**single**
gear_cat				
multiple	2.67	-2.36	14.33	16.03

	dna_advantage		union	
marker_cat	**multiple**	**single**	**multiple**	**single**
gear_cat				
single	30.00	-0.60	103.00	8.00
unspecified	NaN	0.00	NaN	1.00

```
grader.check("q10")
```

`q10` passed! 🌟

Which combination of `gear_cat` and `marker_cat` detects the most species overall (highest mean `union`)? Answer in the markdown cell below

The single gear and multiple marker combination detects the most species overall, with a mean union of 103.0. The second highest is multiple gear and single marker with a mean union of 16.03, followed by multiple gear and multiple marker with 14.33, and finally single gear and single marker with the lowest mean union of 8.0.

Run the cell below to receive credit for all auto graded questions.

```
grader.check_all()
```

```
q10 results: All test cases passed!

q3 results: All test cases passed!

q5 results: All test cases passed!

q6 results: All test cases passed!

q7 results: All test cases passed!

q8 results: All test cases passed!

q9 results: All test cases passed!
```